EXPERIMENT 8

D15A-57

Aim: Design and train a Convolutional Neural Network (CNN) on image datasets

Theory:

1. Dataset Source
The dataset used in this experiment is the Fashion-MNIST dataset. It is publicly available and can be downloaded from: https://github.com/zalandoresearch/fashion-mnist. The dataset is provided in IDX file format and is widely used as a benchmark for image classification tasks.

2. Dataset Description
Fashion-MNIST is a dataset of grayscale images representing different categories of clothing items. It consists of 70,000 images, divided into 60,000 training samples and 10,000 testing samples. Each image is of size 28×28 pixels and belongs to one of 10 classes, including T-shirt/top, trouser, pullover, dress, coat, sandal, shirt, sneaker, bag, and ankle boot. The pixel values range from 0 to 255, representing the intensity of grayscale. The target variable is the class label corresponding to each image. The dataset is balanced, with an equal number of images in each class, making it suitable for classification tasks.

3. Mathematical Formulation of the Algorithm
A Convolutional Neural Network (CNN) is used for image classification. The convolution operation is defined as:

$$Z_{i,j}^{(k)} = \sum_m \sum_n X_{i+m,j+n} \cdot W_{m,n}^{(k)} + b^{(k)}$$

where X is the input image, $W^{(k)}$ is the filter for the kth feature map, and $b^{(k)}$ is the bias term. The output is passed through an activation function such as ReLU:

$$f(x) = \max(0, x)$$

4. Algorithm Limitations
CNNs require a large amount of data and computational resources for effective training. They may overfit when trained on small datasets. The model performance is sensitive to hyperparameter choices such as learning rate, number of layers, and filter sizes. CNNs also lack interpretability compared to traditional machine learning models. Additionally, they may not perform well if the dataset contains significant noise or irrelevant features.

5. Methodology / Workflow
The experiment begins with loading the Fashion-MNIST dataset and preprocessing the data by normalizing pixel values and reshaping the images into the required input format. The dataset is then divided into training and testing sets. A CNN architecture is designed consisting of convolutional layers, activation functions, pooling layers, and fully connected layers. The model

is trained using the training dataset with an optimizer such as Adam. During training, the network learns features automatically from the images. After training, the model is evaluated on the test dataset to measure its performance. The results are analyzed using accuracy and confusion matrix.

6. Performance Analysis

The performance of the CNN model is evaluated using accuracy as the primary metric. Accuracy is defined as the ratio of correctly predicted samples to the total number of samples. For the Fashion-MNIST dataset, the model typically achieves an accuracy between 85% and 92%. A confusion matrix is used to further analyze the classification performance by showing the number of correct and incorrect predictions for each class. This helps in identifying which classes are being misclassified and understanding the strengths and weaknesses of the model.

7. Hyperparameter Tuning

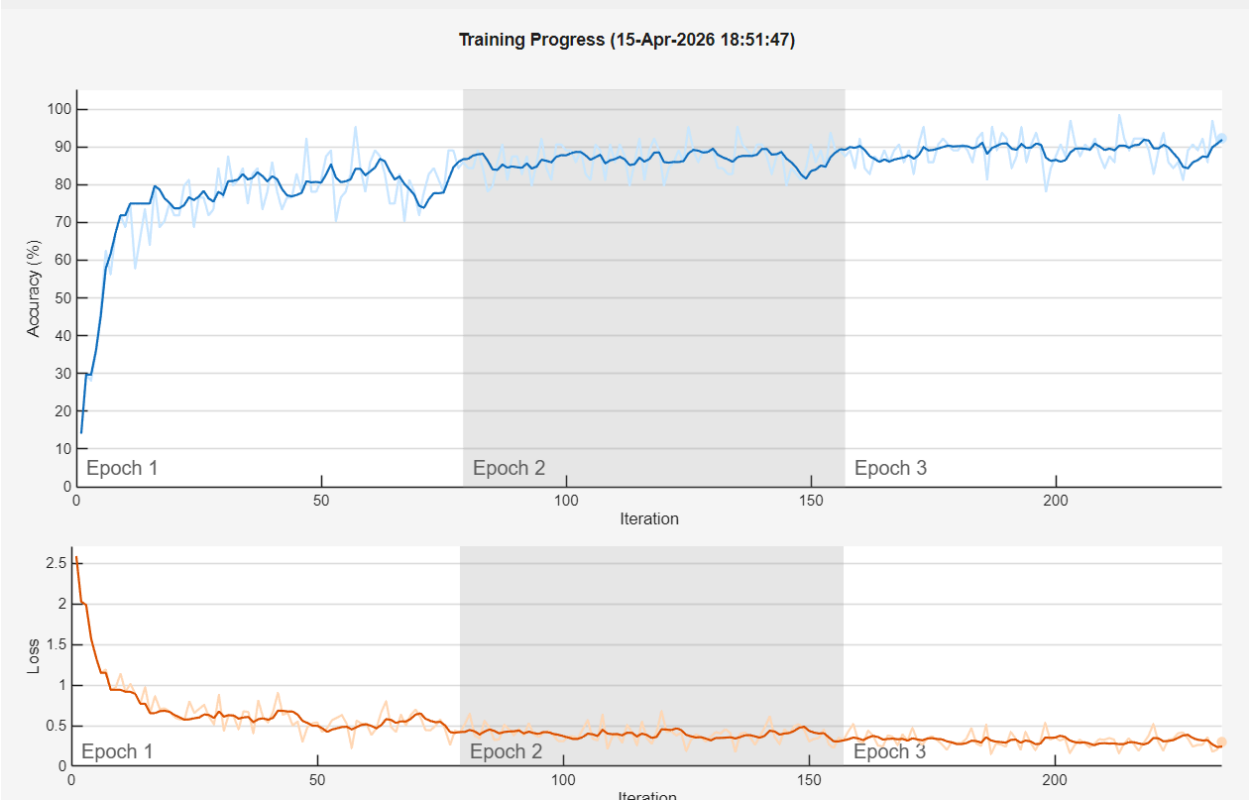
Hyperparameter tuning is performed to improve the performance of the CNN model. Parameters such as the number of convolutional layers, number of filters, kernel size, learning rate, batch size, and number of epochs are adjusted. For example, increasing the number of filters improves feature extraction, while adjusting the learning rate helps in faster convergence. The Adam optimizer is commonly used due to its adaptive learning rate capability. By experimenting with different combinations of hyperparameters, the model performance is improved, leading to higher accuracy and better generalization on unseen data.

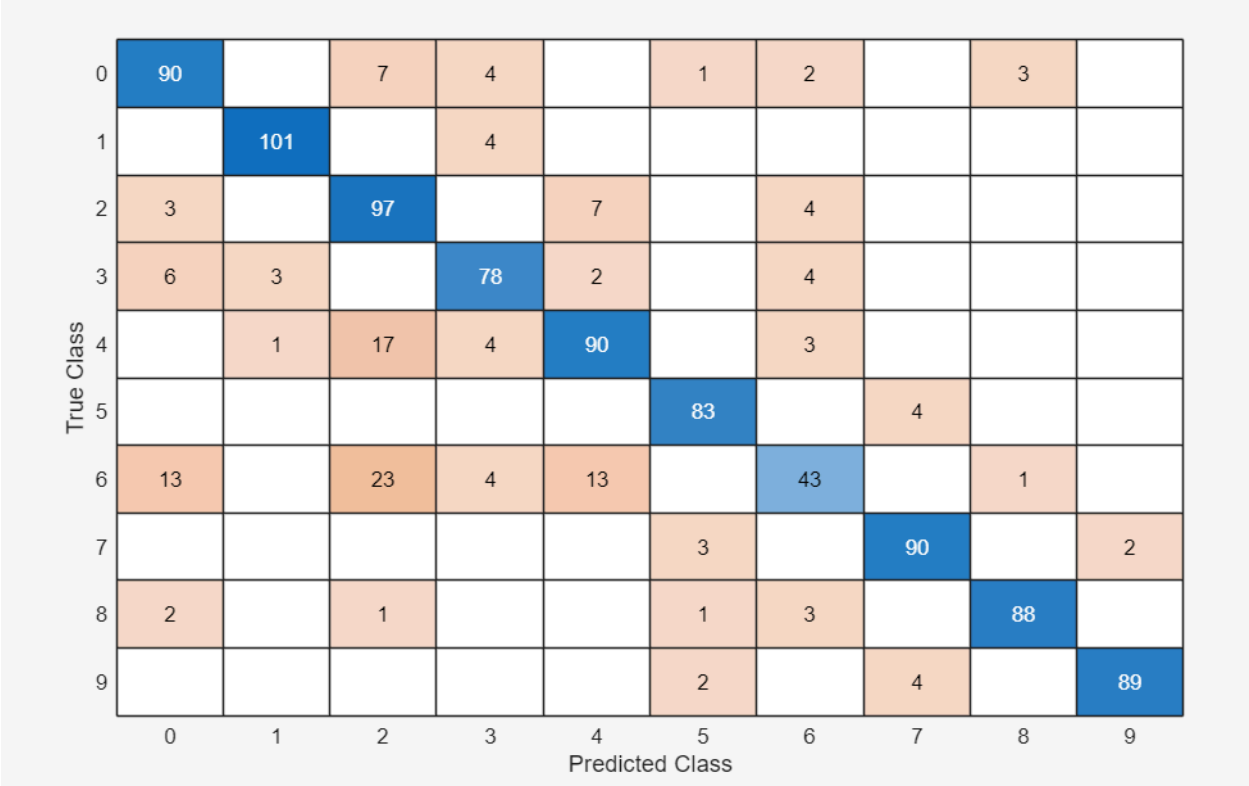
Code:

```
clc;
clear;
close all;

% Load Fashion-MNIST dataset
[XTrain, YTrain] = loadFashionMNIST('train-images-idx3-ubyte','train-labels-idx1-ubyte');
[XTest, YTest] = loadFashionMNIST('t10k-images-idx3-ubyte','t10k-labels-idx1-ubyte');

% Reshape data
XTrain = reshape(XTrain,28,28,1,[]);
XTest = reshape(XTest,28,28,1,[]);

% Convert labels
YTrain = categorical(YTrain);
YTest = categorical(YTest);

% Reduce dataset size (for faster training)
XTrain = XTrain(:,:,:,1:5000);
YTrain = YTrain(1:5000);

XTest = XTest(:,:,:,1:1000);
YTest = YTest(1:1000);

% CNN Architecture
layers = [
    imageInputLayer([28 28 1])

    convolution2dLayer(3,8,'Padding','same')
    batchNormalizationLayer
    reluLayer

    maxPooling2dLayer(2,'Stride',2)

```

```
    convolution2dLayer(3,16,'Padding','same')
    batchNormalizationLayer
    reluLayer

    fullyConnectedLayer(10)
    softmaxLayer
    classificationLayer
];

% Training options
options = trainingOptions('adam', ...
    'MaxEpochs',3, ...
    'MiniBatchSize',64, ...
    'Plots','training-progress', ...
    'Verbose',true);
% Train network
net = trainNetwork(XTrain, YTrain, layers, options);
% Predictions
YPred = classify(net, XTest);
% Accuracy
accuracy = mean(YPred == YTest);
disp(['Accuracy: ', num2str(accuracy)]);
% Confusion Matrix
figure;
confusionchart(YTest, YPred);
% Show sample predictions
figure;
for i = 1:9
    subplot(3,3,i);
    imshow(XTest(:,:,:,i));
    title(string(YPred(i)));
end
```

Output:


Training Progress (15-Apr-2026 18:51:47)
Accuracy (%)
Iteration
Epoch 1
Epoch 2
Epoch 3
Loss
Iteration
Epoch 1
Epoch 2
Epoch 3


True Class
Predicted Class
0 90 7 4 1 2 3
1 101 4
2 3 97 7 4
3 6 3 78 2 4
4 1 17 4 90 3
5 83 4
6 13 23 4 13 43 1
7 3 90 2
8 2 1 1 3 88
9 2 4 89
0 1 2 3 4 5 6 7 8 9

```
Training on single CPU.
Initializing input data normalization.
|=========================================================================================|
|  Epoch  |  Iteration  |  Time Elapsed  |  Mini-batch  |  Mini-batch  |  Base Learning  |
|         |             |   (hh:mm:ss)   |   Accuracy   |     Loss     |      Rate       |
|=========================================================================================|
|       1 |           1 |       00:00:05 |       14.06% |       2.5881 |          0.0010 |
|       1 |          50 |       00:00:09 |       81.25% |       0.5402 |          0.0010 |
|       2 |         100 |       00:00:11 |       89.06% |       0.3089 |          0.0010 |
|       2 |         150 |       00:00:13 |       90.62% |       0.3078 |          0.0010 |
|       3 |         200 |       00:00:16 |       87.50% |       0.3593 |          0.0010 |
|       3 |         234 |       00:00:17 |       92.19% |       0.3031 |          0.0010 |
|=========================================================================================|
Training finished: Max epochs completed.
Accuracy: 0.849
```

Conclusion:

The design and implementation of a Convolutional Neural Network for image classification using the Fashion-MNIST dataset demonstrates the effectiveness of deep learning techniques in recognizing patterns within image data. The model successfully learned important visual features and was able to classify different categories of clothing with good accuracy. The training process showed steady improvement, indicating that the network was able to generalize from the data.

The results highlight the strength of CNNs in automatically extracting features, reducing the need for manual feature engineering. The evaluation using accuracy and confusion matrix provided insights into the model's performance and areas where misclassification occurred, particularly among visually similar classes.

Overall, Convolutional Neural Networks prove to be highly suitable for image classification tasks. The performance can be further enhanced by optimizing hyperparameters, increasing dataset size, and using more advanced architectures.